FORTISAID LAB 4

Configuring and Securing Azure Container Registry (ACR) and Azure Kubernetes Service (AKS)

Region: West US 2
Resource Group: `FortisAid-Lab04`
AKS Cluster: `FortisAid-AKS`
ACR Name: `fortisbabs`

Goal:

- Build and store container image in ACR
- Deploy AKS cluster (Free tier)
- Attach ACR using managed identity (RBAC)
- Deploy external and internal LoadBalancer services
- Validate controlled exposure

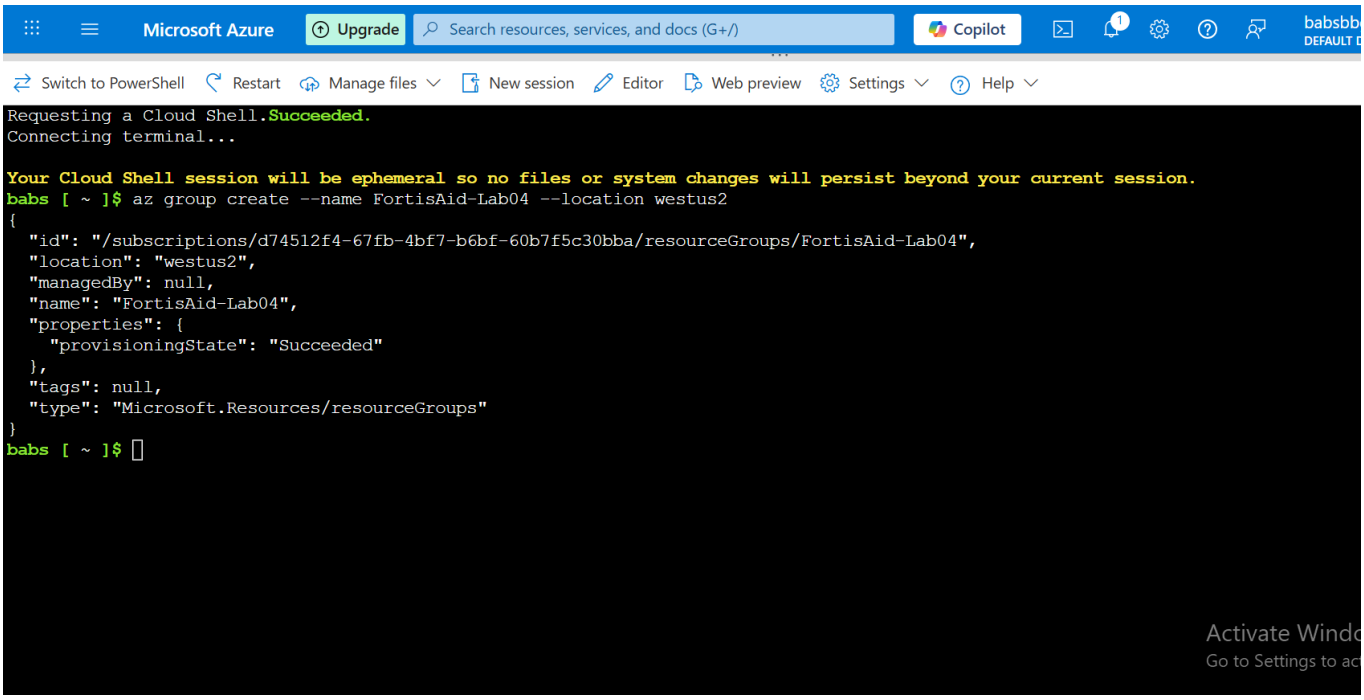
Exercise 1 — Create Azure Container Registry

Step 1 — Create Resource Group

```
az group create \
  --name FortisAid-Lab04 \
  --location westus2
```

Verification

```
az group show -n FortisAid-Lab04 -o table
```



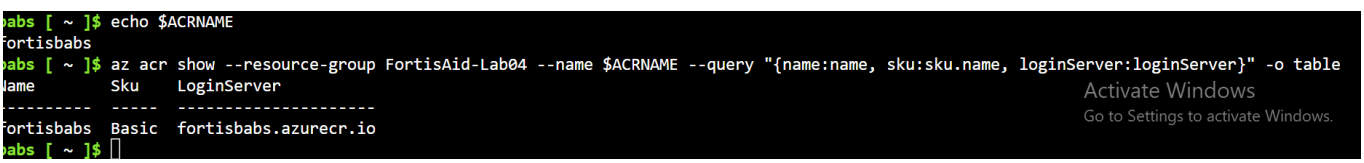
Cloud Shell showing FortisAid-Lab04 created in West US 2

Step 2 — Create ACR (Basic SKU)

```
az acr create \
  --resource-group FortisAid-Lab04 \
  --name fortisbabs \
  --sku Basic \
  --admin-enabled true
```

Verification

```
az acr show -n fortisbabs --query "{Name:name, SKU:sku.name, LoginServer:loginServer}" -o table
```



[Screenshot 2: ACR Overview showing fortisbabs.azurecr.io and SKU = Basic]

Exercise 2 — Build and Push Image

Step 3 — Create Dockerfile

```
echo FROM nginx > Dockerfile
```

[Screenshot 3: Dockerfile content showing FROM nginx]

Step 4 — Build and Push Image to ACR

```
az acr build \
  --resource-group FortisAid-Lab04 \
  --registry fortisbabs \
  --image sample/nginx:v1 \
  --file Dockerfile \
  .
```

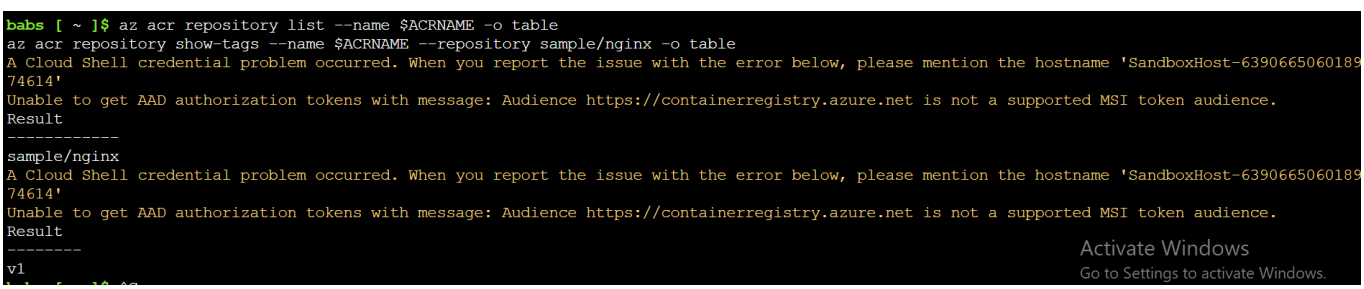
[Screenshot 4: az acr build successful completion]

Step 5 — Verify Repository and Tag

```
az acr repository list --name fortisbabs -o table
az acr repository show-tags --name fortisbabs --repository sample/nginx -o table
```

Expected Output:

```
sample/nginx
v1
```



[Screenshot 5: Repository list showing sample/nginx]
[Screenshot 6: Tag list showing v1]

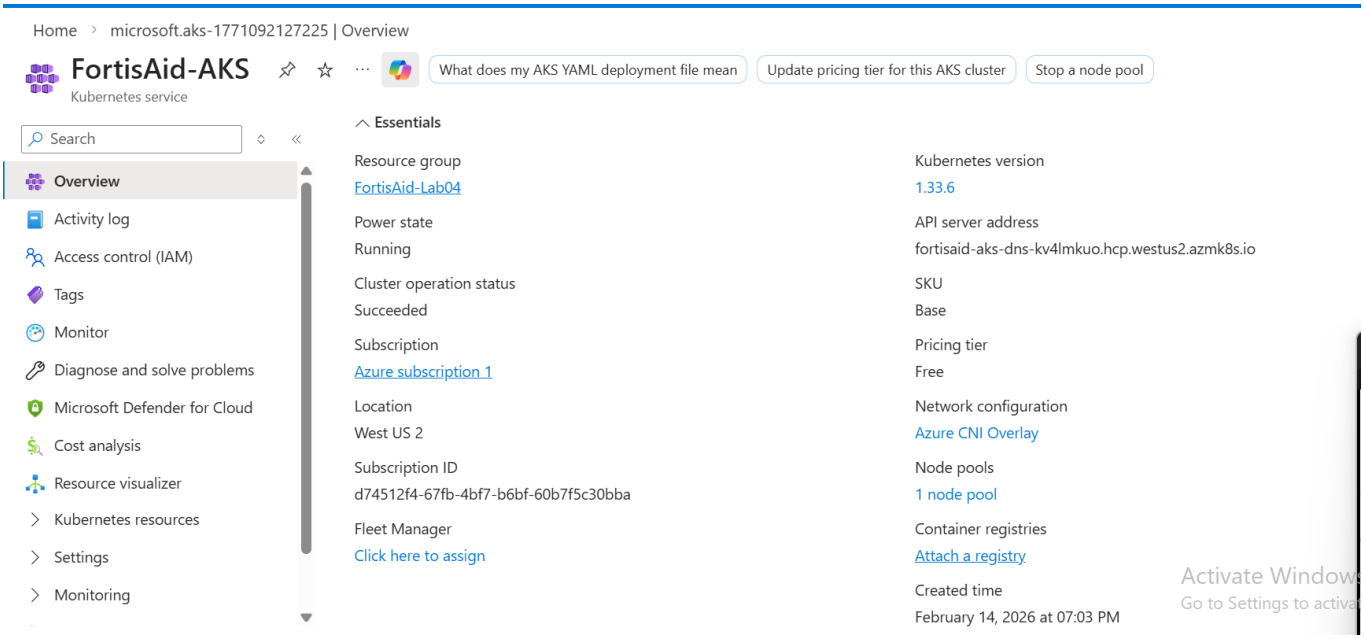
Exercise 3 — Deploy AKS Cluster

Step 6 — Create AKS Cluster (Portal)

Configuration that worked:

- Preset: Dev/Test
- Pricing tier: Free
- Region: West US 2
- Cluster name: FortisAid-AKS
- Kubernetes version: Default
- Networking: Azure CNI Overlay
- Node pool:
  - Mode: System
  - OS: Ubuntu
  - Size: Standard_D2s_v6
  - Node count: 1
  - Availability zones: None

Cluster deployment succeeded.



FortisAid-AKS Overview showing Running and Succeeded status

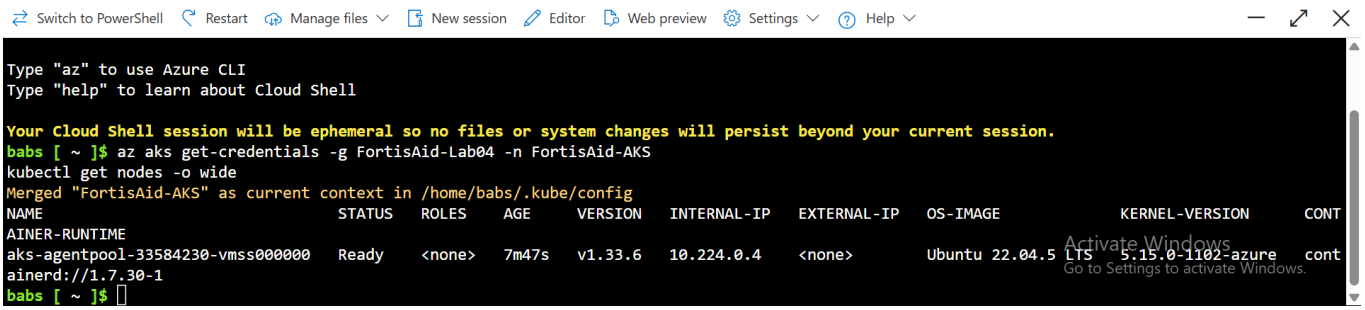
Step 7 — Connect kubectl

```
az aks get-credentials \
  --resource-group FortisAid-Lab04 \
  --name FortisAid-AKS \
  --overwrite-existing
```

```
kubectl get nodes
```

Expected:

```
STATUS: Ready
```

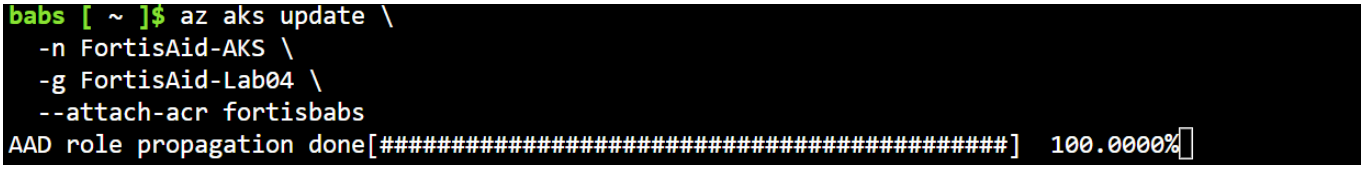


kubectl get nodes showing Ready status

Exercise 4 — Attach ACR to AKS

Step 8 — Attach Registry

```
az aks update \
  -n FortisAid-AKS \
  -g FortisAid-Lab04 \
  --attach-acr fortisbabs
```



az aks update --attach-acr successful output

Exercise 5 — Deploy External Service

Step 9 — Deploy External LoadBalancer

Image used:

```
fortisbabs.azurecr.io/sample/nginx:v1
```

```
kubectl apply -f nginxexternal.yaml
```

```
kubectl get svc nginxexternal
```

External IP assigned.

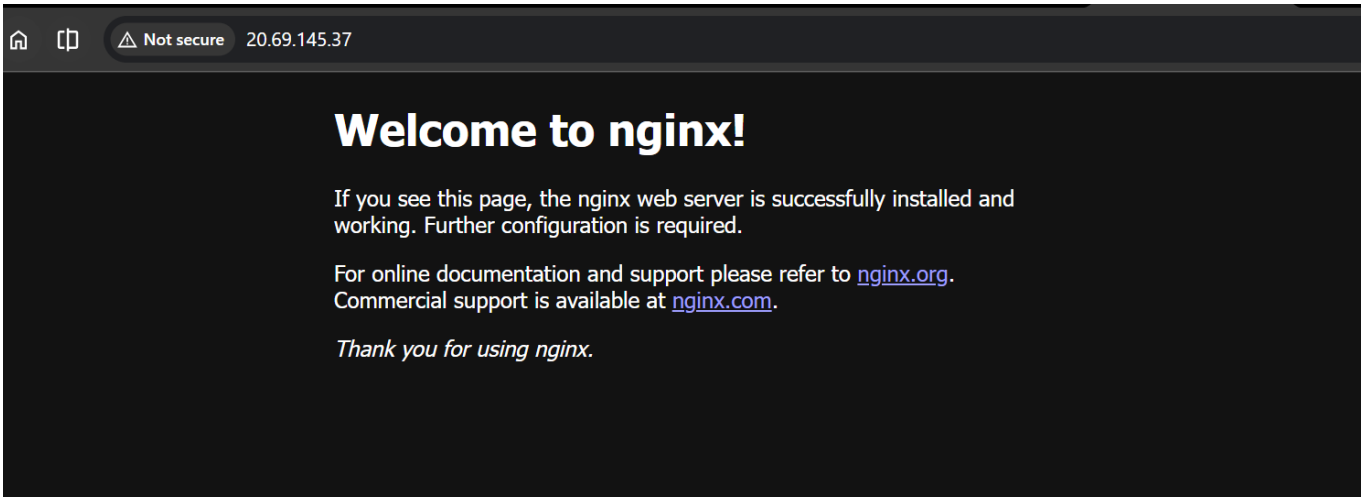
```
babs [ ~ ]$ kubectl get svc nginxexternal -w
NAME            TYPE           CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
nginxexternal   LoadBalancer   10.0.166.91   20.69.145.37   80:31638/TCP   30s
```

kubectl get svc showing external public IP

Open browser:

```
http://<EXTERNAL-IP>
```

Nginx page loads successfully.



Browser showing nginx default page via public IP

Exercise 6 — Deploy Internal Service

Step 10 — Deploy Internal LoadBalancer

```
kubectl apply -f nginxinternal.yaml
```

```
kubectl get svc nginxinternal
```

Internal IP assigned (10.x.x.x).

```
babs [ ~ ]$ kubectl get svc nginxinternal
kubectl get pods
NAME            TYPE           CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
nginxinternal   LoadBalancer   10.0.60.249   10.224.0.5    80:31016/TCP   39s
NAME                              READY   STATUS    RESTARTS   AGE
nginxexternal-7f5fb9b45f-rvvtq    1/1     Running   0          22m
nginxinternal-f8746d8fc-c2swk     1/1     Running   0          40s
babs [ ~ ]$
```

kubectl get svc showing private internal IP

Step 11 — Validate Internal Access from Pod

```
POD=$(kubectl get pods -l app=nginxexternal -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it $POD -- sh
```

Inside pod:

```
curl http://nginxinternal
```

HTML returned successfully.

```
babs [ ~ ]$ kubectl run fortisaid-curl --rm -it --image=curlimages/curl --restart=Never -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the
command prompt.
If you don't see a command prompt, try pressing enter.
~ $ curl -sS http://nginxinternal
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
~ $
```

curl output showing nginx HTML from internal service

Reflection — Issues Encountered & Resolutions

1. AKS System Node Pool Requirement

Error:

```
Creating AKS cluster without a system mode node pool is not permitted
```

Resolution:
Changed node pool mode to **System**.

2. Availability Zone Restriction

Error:

```
AvailabilityZoneNotSupported
```

Resolution:
Removed availability zone selection (set to None).

3. kubectl Credential Expiry

After restarting Cloud Shell:

```
You must be logged in to the server
```

Resolution:

```
az aks get-credentials --overwrite-existing
```

4. wget Not Found in Container

Error:

```
wget: not found
```

Resolution:
Used:

```
curl http://nginxinternal
```

Key Learning

This lab demonstrated:

- Secure image storage via ACR
- Managed identity-based ACR pull
- Proper AKS node configuration
- Public vs private service exposure
- Practical troubleshooting in real Azure constraints